ChefKart

Home Chef Booking Platform

Project Report

Project Name	ChefKart — Home Chef Booking Platform
Technology Stack	React 18 · Node.js · Express.js · MongoDB · JWT · Razorpay
Architecture	MVC Pattern | RESTful API | MERN Stack
Repository	https://github.com/Amrutha123-bot/ChefKart

Table of Contents

1. Introduction
2. Project Overview & Objectives
3. System Requirements
4. Project Architecture
5. ER Diagram & Database Design
6. Key Features
7. Roles and Responsibilities
8. User Flow
9. MVC Pattern
10. Backend Development
11. Database Development
12. Frontend Development
13. API Endpoints
14. Setup & Configuration
15. Deployment Guide
16. Conclusion

1. Introduction

ChefKart is a full-stack Home Chef Booking Platform built on the MERN stack (MongoDB, Express.js, React 18, Node.js). The platform connects home-based chefs with customers who wish to book personalised, home-cooked meals or cooking services. By digitising the discovery, booking, and payment process, ChefKart eliminates the friction typically associated with hiring private or home chefs.

The application supports three distinct user roles — Customer, Chef, and Admin — each with a dedicated dashboard and tailored feature set. Customers can browse chef profiles, filter by cuisine or location, book sessions, make payments via Razorpay, and leave reviews. Chefs can manage their profile, availability, and bookings. Administrators oversee the entire platform through a centralised management panel.

1.1 Purpose

The purpose of ChefKart is to provide a trustworthy, convenient, and efficient marketplace for home chef services. It aims to empower home chefs with technology to reach a wider audience, while giving customers access to quality, personalised culinary experiences.

1.2 Scope

- User registration, login, and role-based authentication using JWT.
- Chef profile management with cuisine specialisation, pricing, and availability.
- Advanced chef search and filtering by cuisine, rating, price, and location.
- End-to-end booking system with date, time, and duration selection.
- Razorpay payment gateway integration for secure online transactions.
- Review and star-rating system for quality assurance.
- Separate dashboards for Customer, Chef, and Admin roles.
- Admin controls for user/chef management and platform oversight.
- Responsive design supporting desktop and mobile browsers.
- Database seeding with sample data for demonstration.

2. Project Overview & Objectives

ChefKart addresses a growing demand for personalised culinary services at home. The platform bridges the gap between skilled home chefs and food-loving customers through a seamless digital interface.

2.1 Problem Statement

Finding reliable, skilled home chefs is a manual, word-of-mouth driven process with no standardised discovery, vetting, or payment mechanism. Chefs lack a platform to showcase their skills and reach potential clients. ChefKart solves both sides of this problem with a structured marketplace.

2.2 Objectives

1. Build a scalable MERN stack web application with role-based access control.
2. Provide a searchable and filterable chef directory for customers.
3. Enable real-time booking management for both customers and chefs.
4. Integrate a secure payment gateway (Razorpay) for hassle-free transactions.
5. Establish a trustworthy review and rating ecosystem.
6. Give administrators full visibility and control over platform activity.
7. Deploy the application for public access using cloud platforms.

2.3 Tech Stack Summary

Layer	Technology
Frontend	React 18, Vite, React Router v6
Styling	Pure CSS (custom design system)
HTTP Client	Axios
Backend	Node.js, Express.js
Database	MongoDB, Mongoose ODM
Auth	JWT (JSON Web Tokens) + bcryptjs
Payments	Razorpay
Icons	React Icons
Toasts	React Hot Toast
Deployment	Render (backend), Vercel (frontend)

3. System Requirements

3.1 Software Requirements

Component	Specification
Operating System	Windows 10/11, macOS, or Linux
Node.js	v18 or above
npm	v9 or above
MongoDB	v6+ (local) or MongoDB Atlas (cloud)
React	v18 (via Vite build tool)
Browser	Google Chrome / Firefox (latest version)
Git	Latest version for version control
VS Code	Recommended IDE with ESLint and Prettier extensions
Postman	For API testing and development
Razorpay Account	Test/live keys for payment gateway integration

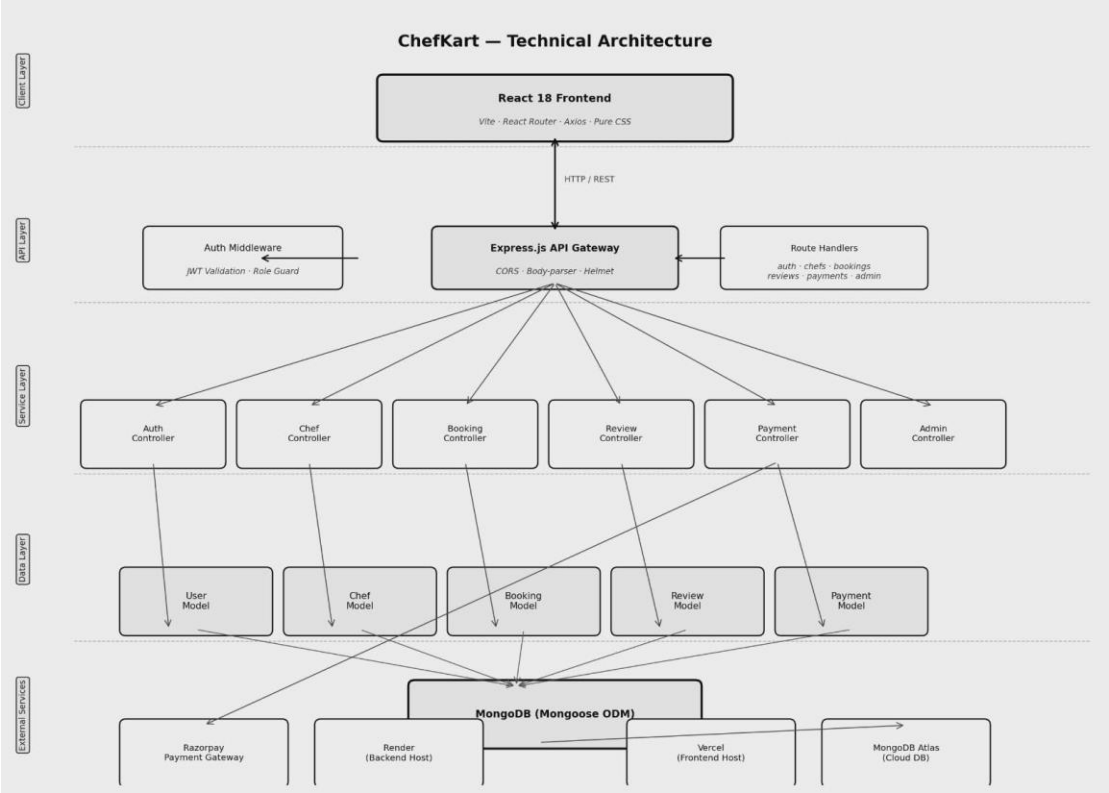
3.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD equivalent	Intel Core i5 (8th Gen+) or better
RAM	4 GB	8 GB or more
Storage	500 MB free space	2 GB free space
Display	1280×720	1920×1080 or higher
Internet	Required for MongoDB Atlas, Razorpay, deployment	Broadband connection

4. Project Architecture

4.1 High-Level Architecture

ChefKart follows a client-server architecture with clear separation between the React frontend and the Express/Node.js backend. The backend exposes a RESTful API consumed by the frontend via Axios. MongoDB serves as the data persistence layer, accessed through the Mongoose ODM. Authentication is handled using JWT tokens stored on the client and validated by middleware on the backend.



4.2 Full Project Folder Structure

Path / File	Purpose
chefkart/	Root project directory
backend/	All server-side code
backend/config/	Database and environment configuration
backend/controllers/	Business logic handlers for each resource
backend/controllers/authController.js	Registration, login, password management
backend/controllers/bookingController.js	Create, update, cancel, and list bookings
backend/controllers/chefController.js	Chef profile CRUD and search
backend/controllers/reviewController.js	Review creation and retrieval
backend/controllers/paymentController.js	Razorpay order creation and verification
backend/controllers/adminController.js	Admin stats, user/chef management

backend/middleware/auth.js	JWT validation and role-based access middleware
backend/models/	Mongoose schemas and models
backend/models/User.js	User schema (customer, chef, admin roles)
backend/models/Chef.js	Chef profile schema with specialisations
backend/models/Booking.js	Booking schema with status tracking
backend/models/Review.js	Review and Payment model schemas
backend/routes/	Express route definitions
backend/seed.js	Database seeding script for demo data
backend/server.js	Main Express server entry point
backend/.env	Environment variables (not committed)
frontend/	All client-side code
frontend/src/components/	Reusable UI components
frontend/src/context/	React context for global auth state
frontend/src/pages/	Page-level components for each route
frontend/src/services/api.js	Centralised Axios API service
frontend/src/App.jsx	Root component with routing setup
frontend/vite.config.js	Vite build and dev server configuration

4.3 Data Flow

Step	Description
1	User interacts with the React frontend (browser).
2	React component calls an Axios function from services/api.js.
3	Axios sends an HTTP request (GET/POST/PUT/DELETE) to the Express backend.
4	Express router matches the endpoint and invokes the relevant controller.
5	auth.js middleware validates the JWT token for protected routes.
6	Controller executes business logic and calls Mongoose model methods.
7	Mongoose queries MongoDB and returns the result to the controller.
8	Controller sends a JSON response back through Express to the frontend.
9	React updates state and re-renders the UI with the new data.

5. ER Diagram & Database Design

The ChefKart database is built on MongoDB with four primary collections: Users, Chefs, Bookings, and Reviews. The schema design leverages MongoDB's document model with ObjectId references for relational links between documents.

5.1 Entities and Key Attributes

Entity / Collection	Key Fields
User	name, email, password (hashed), role [user|chef|admin], phone, avatar, createdAt
Chef	user (ref: User), bio, specialisations [], cuisines [], pricePerHour, location, experience, availability, rating, totalReviews
Booking	customer (ref: User), chef (ref: Chef), date, startTime, duration, totalAmount, status [pending|confirmed|completed|cancelled], paymentStatus
Review	customer (ref: User), chef (ref: Chef), booking (ref: Booking), rating (1-5), comment, createdAt
Payment	user (ref: User), booking (ref: Booking), razorpayOrderId, razorpayPaymentId, amount, status [created|paid|failed]

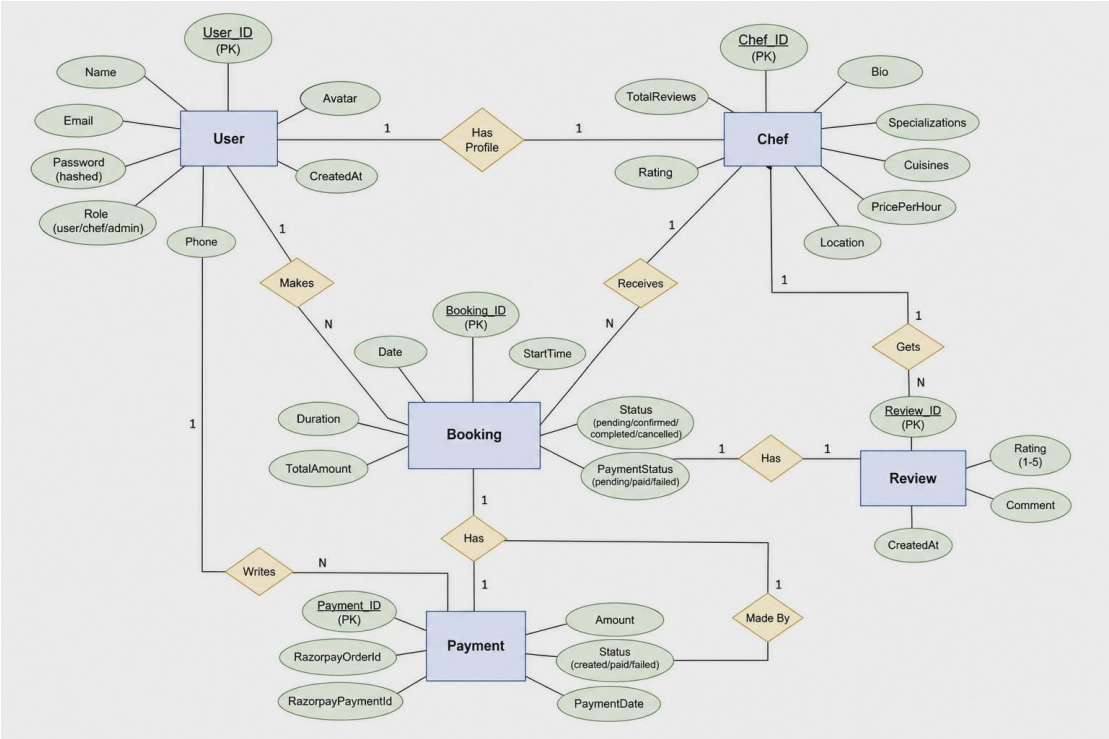
5.2 Relationships

Relationship	Type	Description
User — Chef	One-to-One	A user with role "chef" has one Chef profile document
User — Booking	One-to-Many	A customer (User) can place multiple bookings
Chef — Booking	One-to-Many	A chef can receive multiple bookings
Booking — Review	One-to-One	Each completed booking can have one review
User — Review	One-to-Many	A user can leave multiple reviews for different chefs
Chef — Review	One-to-Many	A chef can receive multiple reviews
Booking — Payment	One-to-One	Each booking has one associated payment record

5.3 Primary and Foreign Keys

Collection	Primary Key	Foreign Keys (References)
User	_id (ObjectId)	None
Chef	_id (ObjectId)	user → User._id
Booking	_id (ObjectId)	customer → User._id, chef → Chef._id
Review	_id (ObjectId)	customer → User._id, chef → Chef._id, booking → Booking._id

Payment	_id (ObjectId)	user → User._id, booking → Booking._id



6. Key Features

User Registration & Login (JWT): Secure registration and login for all roles. Passwords are hashed with bcryptjs. JWT tokens are issued on login and validated on every protected request via middleware.

Role-Based Access Control: Three roles — User (Customer), Chef, and Admin — each with distinct permissions and dedicated dashboards. Role is stored in the JWT payload and enforced server-side.

Chef Discovery — Browse & Filter: Public chef listing page with real-time filters for cuisine type, minimum rating, maximum price per hour, location, and text search. Supports pagination.

Chef Profile Pages: Detailed public profile for each chef including bio, specialisations, cuisine list, pricing, experience years, location, average rating, and a gallery of reviews.

Booking System: Customers select a date, start time, and duration to book a chef. The system calculates the total amount and creates a pending booking record.

Chef Booking Management: Chefs receive booking requests and can accept or reject them. Status transitions: pending → confirmed → completed, or cancelled.

Razorpay Payment Integration: Seamless payment flow: a Razorpay order is created on the backend, the customer completes payment in a Razorpay popup, and the payment signature is verified server-side.

Review & Star Rating System: After a booking is completed, customers can leave a star rating (1-5) and a written review. Chef average ratings are recalculated automatically.

Customer Dashboard: Customers can view upcoming and past bookings, track booking status, initiate payments, and cancel pending bookings.

Chef Dashboard: Chefs can edit their profile (bio, cuisines, pricing, availability), view incoming bookings, and update booking statuses.

Admin Dashboard: Admins can view platform statistics (total users, chefs, bookings, revenue), manage all users (activate/deactivate), verify chef profiles, and monitor all bookings.

Responsive Design: Fully mobile-friendly layout built with pure CSS, ensuring a consistent experience across desktop, tablet, and mobile devices.

Database Seeding: A seed.js script populates the database with an admin, a sample user, and six diverse chef profiles for immediate demonstration.

7. Roles and Responsibilities

7.1 Customer (User Role)

Responsibility	Description
Registration & Profile	Create an account, provide personal details, manage profile information.
Chef Discovery	Browse the chef listing, apply filters, and view detailed chef profiles.
Booking	Select service date, time, and duration; submit a booking request.
Payment	Complete payment for confirmed bookings via Razorpay.
Booking Management	View booking history, track status, and cancel pending bookings.
Review & Rating	Submit star ratings and written reviews after completed bookings.
Communication	Contact support or chefs regarding booking queries.

7.2 Chef Role

Responsibility	Description
Profile Setup	Complete chef profile: bio, cuisine specialisations, pricing, experience, location.
Availability	Set and update working availability for customer booking reference.
Booking Management	Review incoming booking requests; confirm or reject based on availability.
Status Updates	Mark confirmed bookings as completed upon service delivery.
Review Monitoring	View customer reviews and ratings received on the profile.

7.3 Administrator Role

Responsibility	Description
Platform Oversight	Monitor overall platform statistics: users, chefs, bookings, revenue.
User Management	View all registered users; activate or deactivate accounts as needed.
Chef Verification	Review and verify chef profiles to ensure quality and authenticity.
Booking Monitoring	Access all bookings across the platform for oversight and dispute resolution.

Content Moderation	Ensure reviews and chef listings comply with platform guidelines.
System Administration	Manage environment configuration, database integrity, and deployments.

8. User Flow

8.1 Customer Flow

1. Visit the landing page (/) and browse featured chefs.
2. Navigate to /chefs and apply filters (cuisine, location, price, rating).
3. Click on a chef card to view the detailed profile (/chefs/:id).
4. Register or log in (/register or /login) to proceed with a booking.
5. Select a booking date, start time, and duration; submit the booking form.
6. Booking is created with status "pending" awaiting chef confirmation.
7. Once the chef confirms, the customer receives a notification and pays via Razorpay.
8. After the service is delivered, the booking status becomes "completed".
9. Customer submits a star rating and written review for the chef.
10. Customer can track all bookings in the Dashboard (/dashboard).

8.2 Chef Flow

1. Register as a Chef (/register) or log in (/login).
2. Complete the chef profile via the Chef Dashboard (/chef-dashboard).
3. Set cuisine specialisations, pricing, experience, and availability.
4. Receive incoming booking requests on the dashboard.
5. Review booking details and confirm or reject the request.
6. On the agreed date, deliver the cooking service to the customer.
7. Mark the booking as "completed" on the dashboard.
8. View customer reviews and ratings received on the profile.

8.3 Admin Flow

1. Log in with admin credentials (/login).
2. Access the Admin Panel (/admin) with the platform overview dashboard.
3. Review platform stats: total users, chefs, bookings, and revenue.
4. Manage user accounts — view all users, activate or deactivate accounts.
5. Verify new chef registrations to ensure platform quality.
6. Monitor all bookings across the platform for oversight.
7. Handle disputes or escalations reported by customers or chefs.

9. MVC Architectural Pattern

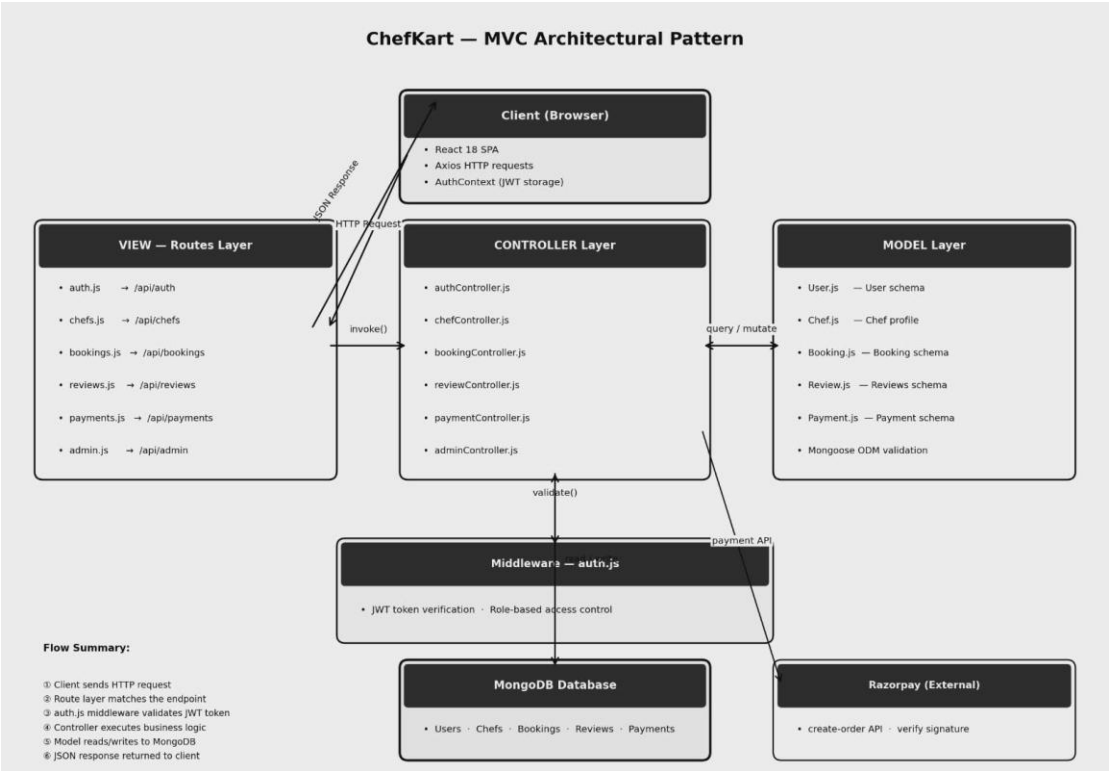
ChefKart follows the Model-View-Controller (MVC) architectural pattern on the backend, ensuring a clean separation of concerns between data handling, business logic, and request routing.

9.1 Layer Descriptions

Layer	Implementation	Responsibility
Model	Mongoose schemas in backend/models/	Define data structure, validation rules, and MongoDB interactions. Exports Mongoose models used by controllers.
View	Express route files in backend/routes/	Define API endpoint paths and HTTP methods. Map incoming requests to the appropriate controller function. Act as the interface between client and server.
Controller	Handler functions in backend/controllers/	Contain all business logic. Validate input, call model methods, handle errors, and return JSON responses to the client.

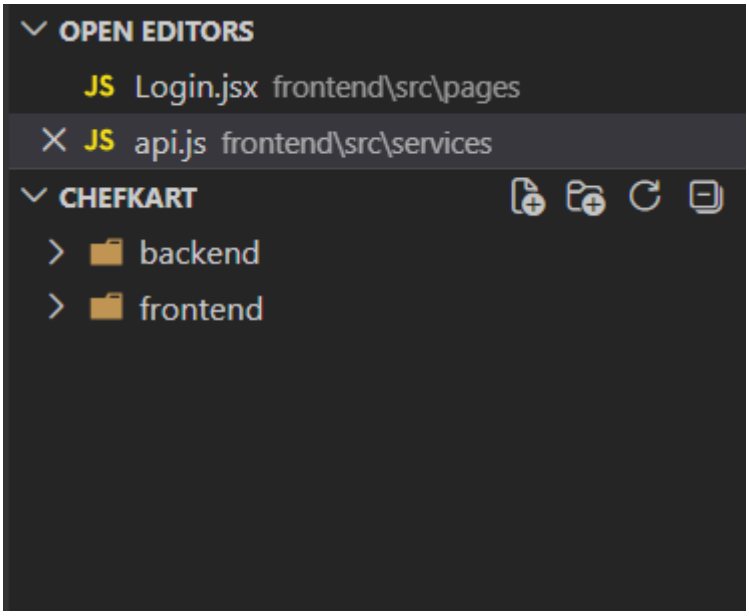
9.2 Advantages in ChefKart

Advantage	How It Applies to ChefKart
Separation of Concerns	Routes, logic, and data are in distinct files, making the codebase easy to navigate.
Scalability	New resources (e.g., Notifications) require only a new model, controller, and route file.
Reusability	Auth middleware and controller helpers are reused across all protected routes.
Testability	Controllers and models can be unit-tested independently of the HTTP layer.
Team Collaboration	Frontend and backend developers can work in parallel without conflicts.



10. Project SetUp

1. Creating project folder
2. Create a new folder with your .
3. Inside that folder create two new folders.
4. Name one as **frontend**.
5. Name another one as **backend**.
6. Now open that folder in VS Code



11. Backend Development

Backend Reference Video
https://drive.google.com/file/d/1KQ3TcjYJ4Wra6G3Te0pha9ZS-UmY2SVT/view?usp=drivesdk

11.1 Server Entry Point (server.js)

The server.js file initialises the Express application, connects to MongoDB, registers all route handlers, configures CORS with the CLIENT_URL from the environment, and starts listening on the configured PORT. All routes are prefixed with /api.

11.2 Authentication & Middleware (middleware/auth.js)

The auth middleware validates JWT tokens sent in the Authorization header (Bearer token format). It decodes the token using the JWT_SECRET, attaches the user object to req.user, and passes control to the next handler. Role-specific middleware functions (isChef, isAdmin) provide fine-grained access control.

11.3 Controllers Summary

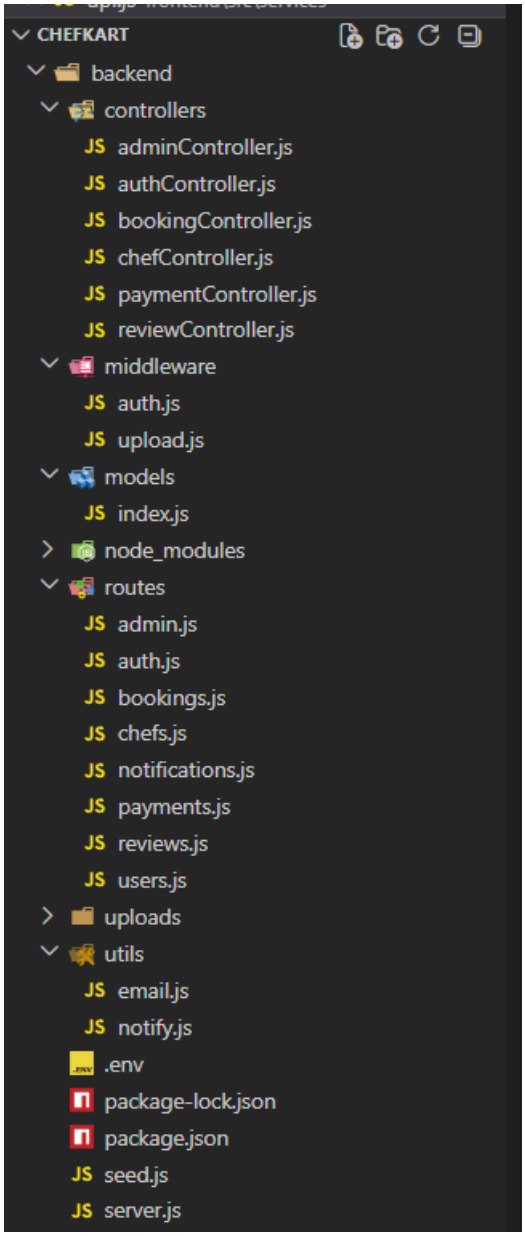
Controller File	Key Functions
authController.js	register, login, getMe, updatePassword
chefController.js	getChefs (with filters/pagination), getChefById, getMyProfile, updateProfile
bookingController.js	createBooking, getMyBookings, getChefBookings, updateBookingStatus, cancelBooking
reviewController.js	createReview, getChefReviews
paymentController.js	createOrder (Razorpay), verifyPayment
adminController.js	getStats, getAllUsers, toggleUserStatus, getAllChefs, verifyChef, getAllBookings

11.4 Route Definitions

Route File	Base Path	Access Level
auth.js	/api/auth	Public + Protected
users.js	/api/users	Protected (User)
chefs.js	/api/chefs	Public + Protected (Chef)
bookings.js	/api/bookings	Protected (User / Chef / Admin)
reviews.js	/api/reviews	Public + Protected (User)
payments.js	/api/payments	Protected
admin.js	/api/admin	Protected (Admin only)

11.5 Environment Variables (.env)

Variable	Description
PORT	Port on which the Express server runs (default: 5000)
MONGO_URI	MongoDB connection string (local or Atlas)
JWT_SECRET	Secret key used to sign and verify JWT tokens
JWT_EXPIRE	JWT token expiry duration (e.g., 7d)
RAZORPAY_KEY_ID	Razorpay API Key ID for payment gateway
RAZORPAY_KEY_SECRET	Razorpay API Key Secret for payment verification
CLIENT_URL	Frontend URL for CORS configuration



12. Database Development

12.1 MongoDB Setup Options

ChefKart supports two MongoDB deployment options:

Option	Setup	Best For
Local MongoDB	Install MongoDB Community and run mongod locally	Development and testing
MongoDB Atlas	Create a free cluster at cloud.mongodb.com	Production and team collaboration

12.2 Mongoose Configuration

8. Install Mongoose: npm install mongoose
9. Create the database connection function in backend/config/.
10. Connect to MongoDB using the MONGO_URI environment variable.
11. Call the connect function in server.js before starting the HTTP server.
12. Handle connection errors gracefully with process.exit(1) on failure.

12.3 Schema Designs

User Schema:

Field	Type	Notes
name	String	Required, trimmed
email	String	Required, unique, lowercase
password	String	Required, hashed with bcryptjs, min 6 chars
role	String	Enum: user | chef | admin, default: user
phone	String	Optional contact number
avatar	String	Profile image URL
createdAt	Date	Auto-set on document creation

```
// ── USER ──────────────────────────────────────────────────────────────
const userSchema = new mongoose.Schema({
  name:     { type: String, required: true, trim: true },
  email:    { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6, select: false },
  role:     { type: String, enum: ['user','chef','admin'], default: 'user' },
  phone:    String,
  address:  String,
  avatar:   { type: String, default: '' },
  isActive: { type: Boolean, default: true },
  chefApplicationStatus: { type: String, enum: ['none','pending','approved','rejected'], default: 'none' },
  chefApplicationData:   { type: Object, default: null },
  resetOtp:        String,
  resetOtpExpires: Date,
}, { timestamps: true });

userSchema.pre('save', async function() {
  if (!this.isModified('password')) return;
  this.password = await bcrypt.hash(this.password, 12);
});
userSchema.methods.comparePassword = async function(p) { return bcrypt.compare(p, this.password); };
userSchema.methods.toJSON = function() { const o = this.toObject(); delete o.password; delete o.resetOtp; return o; };
const User = mongoose.model('User', userSchema);
```

Chef Schema:

Field	Type	Notes
user	ObjectId	ref: User — links to user account
bio	String	Chef biography/introduction
specialisations	[String]	List of dish/service specialisations
cuisines	[String]	Cuisine types (e.g., Indian, Italian)
pricePerHour	Number	Hourly rate in INR
location	String	City or area
experience	Number	Years of experience
availability	Boolean/Object	Working hours and days
rating	Number	Average rating (auto-computed)
totalReviews	Number	Count of reviews received

```
const dishSchema = new mongoose.Schema({
  name:        { type: String, required: true },
  description: String,
  price:       { type: Number, default: 0 },
  prepTime:    String,
  isVeg:       { type: Boolean, default: true },
  image:       { type: String, default: '' }
}, { _id: true });

const chefSchema = new mongoose.Schema({
  userId:      { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, unique: true },
  bio:         { type: String, default: '' },
  cuisines:    [String],
  specialties: [String],
  dishes:      [dishSchema],
  experience:  { type: Number, default: 0 },
  pricing:     { hourly: { type: Number, default: 500 }, daily: { type: Number, default: 0 } },
  availability: [{
    day:  { type: String, enum: ['Mon','Tue','Wed','Thu','Fri','Sat','Sun'] },
    from: String,
    to:   String
  }],
  bookedSlots: [{
    date:      Date,
    timeSlot:  String,
    bookingId: { type: mongoose.Schema.Types.ObjectId, ref: 'Booking' }
  }],
  location:     { type: String, default: '' },
  rating:       { type: Number, default: 0 },
  totalReviews: { type: Number, default: 0 },
  isVerified:   { type: Boolean, default: false },
  isAvailable:  { type: Boolean, default: true },
}, { timestamps: true });
const Chef = mongoose.model('Chef', chefSchema);
```

Booking Schema:

Field	Type	Notes
customer	ObjectId	ref: User
chef	ObjectId	ref: Chef
date	Date	Booking date

startTime	String	Start time (e.g., 18:00)
duration	Number	Session duration in hours
totalAmount	Number	Calculated: pricePerHour × duration
status	String	Enum: pending | confirmed | completed | cancelled
paymentStatus	String	Enum: pending | paid | failed

```
const bookingSchema = new mongoose.Schema({
  userId:          { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  chefId:          { type: mongoose.Schema.Types.ObjectId, ref: 'Chef', required: true },
  date:            { type: Date, required: true },
  timeSlot:        { type: String, required: true },
  duration:        { type: Number, required: true, default: 2 },
  cuisine:         String,
  dishes:          [String],
  specialRequests: String,
  address:         { type: String, required: true },
  status:          { type: String, enum: ['pending','confirmed','rejected','completed','cancelled'], default: 'pending' },
  totalAmount:     { type: Number, required: true },
  paymentStatus:   { type: String, enum: ['pending','paid','refunded'], default: 'pending' },
  paymentId:       String,
  razorpayOrderId: String,
  rejectionReason: String,
  confirmedAt:     Date,
  completedAt:     Date,
}, { timestamps: true });
const Booking = mongoose.model('Booking', bookingSchema);
```

12.4 Database Seeding

The seed.js script populates the database with initial demo data to facilitate testing and demonstration without manual data entry.

Seeded Entity	Details
Admin	admin@chefkart.com / Admin@123
User	user@chefkart.com / User@123
Chef 1	priya@chefkart.com / Chef@123 — South Indian specialist
Chef 2	rahul@chefkart.com / Chef@123 — North Indian and Mughlai
Chef 3	anita@chefkart.com / Chef@123 — Continental and baking
Chef 4-6	Additional sample chefs with varied cuisines and pricing

13. Frontend Development

Frontend Reference Link –

https://drive.google.com/file/d/1Kg_mC4k8d-TfKCQgDR6j_E0sZoQv3bIz/view?usp=drivesdk

13.1 Setup with Vite

13. Navigate to the frontend/ directory.
14. Run: npm install to install all dependencies.
15. Create frontend/.env with VITE_API_URL and VITE_RAZORPAY_KEY.
16. Start the development server: npm run dev
17. The app runs at http://localhost:5173 by default.

13.2 Page Components

Page File	Route	Access	Description
Home.jsx	/	Public	Landing page with hero section and featured chefs
Chefs.jsx	/chefs	Public	Chef listing with search and filter controls
ChefDetail.jsx	/chefs/:id	Public	Individual chef profile with booking form and reviews
Login.jsx	/login	Public	Login form for all roles
Register.jsx	/register	Public	Registration form with role selection
Dashboard.jsx	/dashboard	User only	Customer booking management dashboard
ChefDashboard.jsx	/chef-dashboard	Chef only	Chef profile editor and booking management
AdminDashboard.jsx	/admin	Admin only	Platform stats, user/chef/booking management

13.3 Reusable Components

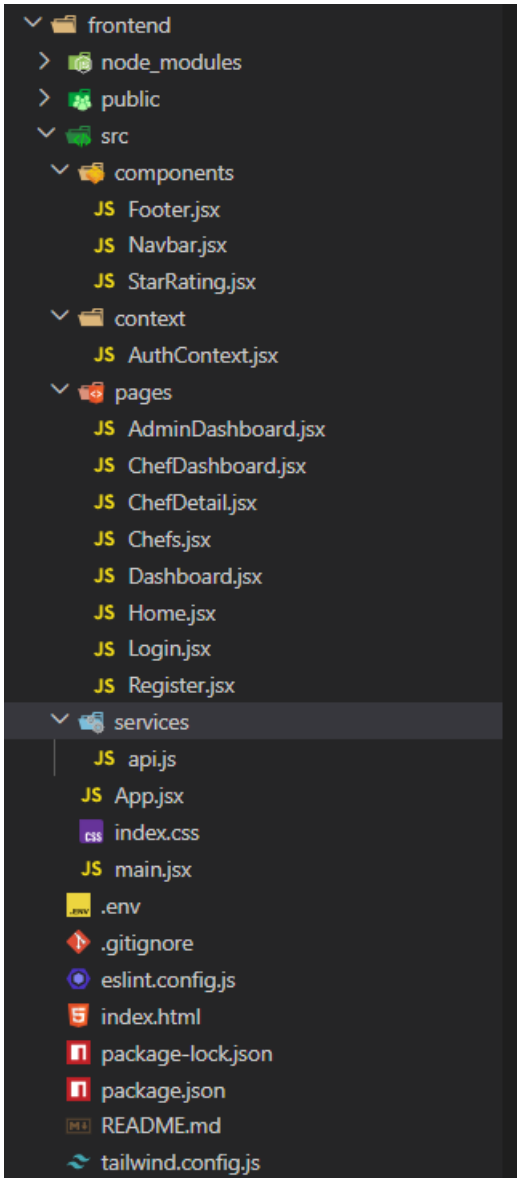
Component	Description
Navbar.jsx	Top navigation bar with role-aware links and logout button
Footer.jsx	Consistent site footer with links and copyright
StarRating.jsx	Reusable interactive star rating component for reviews

13.4 Auth Context (context/AuthContext.jsx)

A React Context provider wraps the entire application to manage global authentication state. It stores the current user object and JWT token, exposes login/logout functions, and provides the auth state to all child components via the useAuth() hook. The token is persisted in localStorage for session continuity.

13.5 API Service (services/api.js)

A centralised Axios instance is configured with the VITE_API_URL base URL. Request interceptors automatically attach the Authorization: Bearer <token> header for authenticated requests. Response interceptors handle 401 errors by redirecting to the login page. All API calls across the application use this shared instance.



14. API Endpoints Reference

14.1 Authentication (/api/auth)

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new user (customer or chef)
POST	/api/auth/login	Public	Login and receive JWT token
GET	/api/auth/me	Protected	Get currently authenticated user
PUT	/api/auth/updatepassword	Protected	Change password

14.2 Chefs (/api/chefs)

Method	Endpoint	Access	Description
GET	/api/chefs	Public	List all chefs; supports ?search, ?cuisine, ?minRating, ?maxPrice, ?location, ?page, ?limit
GET	/api/chefs/:id	Public	Get a specific chef's full profile
GET	/api/chefs/profile/me	Chef only	Get the authenticated chef's own profile
PUT	/api/chefs/profile	Chef only	Update chef profile details

14.3 Bookings (/api/bookings)

Method	Endpoint	Access	Description
POST	/api/bookings	User only	Create a new booking request
GET	/api/bookings/my	User only	Get all bookings made by the customer
GET	/api/bookings/chef	Chef only	Get all bookings received by the chef
PUT	/api/bookings/:id/status	Chef / Admin	Update booking status (confirm/complete/cancel)
PUT	/api/bookings/:id/cancel	User only	Cancel a pending booking

14.4 Reviews, Payments & Admin

Method	Endpoint	Access	Description
POST	/api/reviews	User only	Submit a review for a completed booking
GET	/api/reviews/chef/:id	Public	Get all reviews for a specific chef

POST	/api/payments/create-order	Protected	Create a Razorpay order for a booking
POST	/api/payments/verify	Protected	Verify Razorpay payment signature
GET	/api/admin/stats	Admin only	Get platform-wide statistics
GET	/api/admin/users	Admin only	List all registered users
PUT	/api/admin/users/:id/toggle	Admin only	Activate or deactivate a user account
GET	/api/admin/chefs	Admin only	List all chef profiles
PUT	/api/admin/chefs/:id/verify	Admin only	Verify a chef profile
GET	/api/admin/bookings	Admin only	List all bookings on the platform

15. Setup & Configuration

Step 1 — Install Prerequisites

Tool	Version	Download
Node.js	v18+	https://nodejs.org/en/download
MongoDB	v6+	https://www.mongodb.com/try/download/community
Git	Latest	https://git-scm.com/downloads

Step 2 — Clone the Repository

```
git clone https://github.com/Amrutha123-bot/ChefKart.git

cd ChefKart
```

Step 3 — Backend Setup

18. cd backend
19. npm install
20. cp .env.example .env
21. Edit .env with your MongoDB URI, JWT secret, and Razorpay keys.

Step 4 — Start MongoDB

Option A (Local): Start the MongoDB service on your machine.

Option B (Atlas): Create a free cluster, get the connection string, and update MONGO_URI in .env.

Step 5 — Seed the Database

```
node seed.js
```

This creates the admin, a test user, and 6 sample chef profiles.

Step 6 — Frontend Setup

22. cd ../frontend
23. npm install
24. Create frontend/.env with VITE_API_URL=http://localhost:5000/api and your Razorpay key.

Step 7 — Run the Application

Open two separate terminal windows:

Terminal	Command	Expected Output
Terminal 1 (Backend)	cd backend && npm run dev	MongoDB connected, Server running on port 5000
Terminal 2 (Frontend)	cd frontend && npm run dev	Local: http://localhost:5173/

Demo Login Credentials

Role	Email	Password
Admin	admin@chefkart.com	Admin@123
User	user@chefkart.com	User@123
Chef	priya@chefkart.com	Chef@123
Chef	rahul@chefkart.com	Chef@123
Chef	anita@chefkart.com	Chef@123

15. Deployment Guide

15.1 Backend Deployment — Render.com

25. Push the project to GitHub.
26. Go to https://render.com → New → Web Service.
27. Connect your GitHub repository.
28. Set Root Directory to: backend
29. Set Build Command to: npm install
30. Set Start Command to: node server.js
31. Add all environment variables from .env in the Render dashboard.
32. Update MONGO_URI to your MongoDB Atlas connection string.
33. Click Deploy and copy the Render URL (e.g., https://chefkart-api.onrender.com).

15.2 Frontend Deployment — Vercel.com

34. Go to https://vercel.com → New Project.
35. Import your GitHub repository.
36. Set Root Directory to: frontend
37. Set Framework Preset to: Vite
38. Add environment variables: VITE_API_URL = your Render backend URL + /api
39. Add VITE_RAZORPAY_KEY with your Razorpay key.
40. Click Deploy.

15.3 Post-Deployment Configuration

After deploying both services, update the CLIENT_URL variable in the backend environment on Render to match the Vercel frontend URL. This ensures CORS headers allow requests from the deployed frontend.

15.4 Common Troubleshooting

Issue	Solution
MongoDB connection fails	Verify mongod is running locally, or check Atlas IP whitelist and credentials.
Port already in use	Run: npx kill-port 5000 to free the backend port.
CORS errors in browser	Ensure CLIENT_URL in .env exactly matches the frontend URL with no trailing slash.
Login not working	Re-run node seed.js to reset demo data. Check MongoDB connection logs.
Payment not processing	Verify Razorpay test keys are correctly set in both .env files.

16. Conclusion

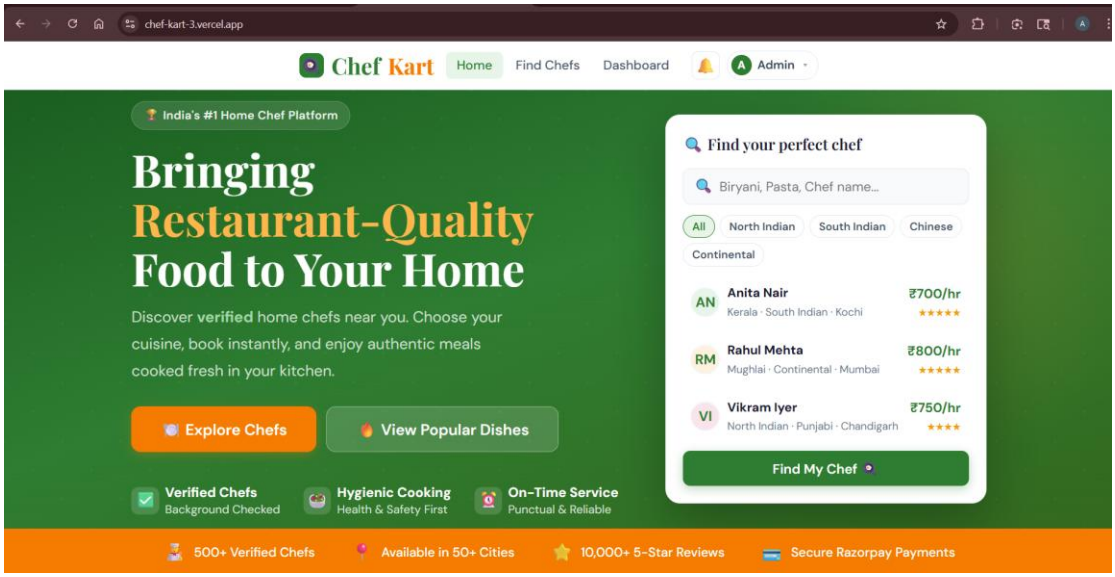
ChefKart successfully demonstrates the development of a complete, production-ready web application using the MERN stack. The platform effectively addresses a real-world problem by connecting home-based chefs with customers seeking personalised culinary services, through a secure, user-friendly, and feature-rich digital marketplace.

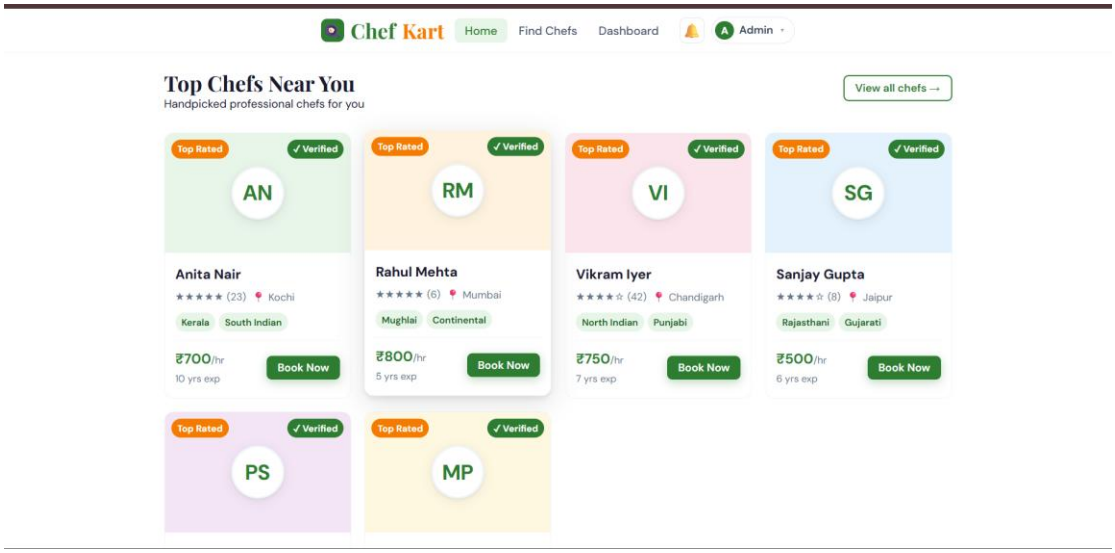
The adoption of role-based access control with three distinct user types — Customer, Chef, and Admin — ensures a tailored experience for each stakeholder. The integration of Razorpay enables reliable, secure payments, while the review and rating system establishes trust and quality assurance on the platform.

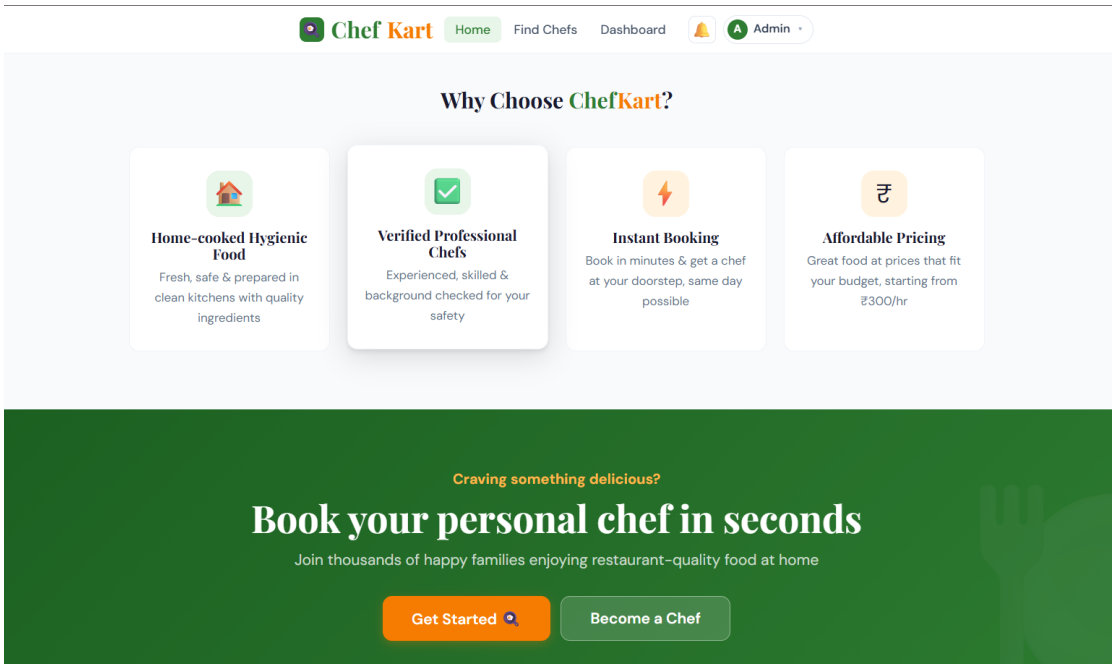
Technically, the MVC architectural pattern, JWT authentication, Mongoose ODM, and Vite-powered React frontend collectively deliver a scalable and maintainable codebase. Deployment on Render and Vercel ensures the application is accessible publicly with minimal infrastructure overhead.

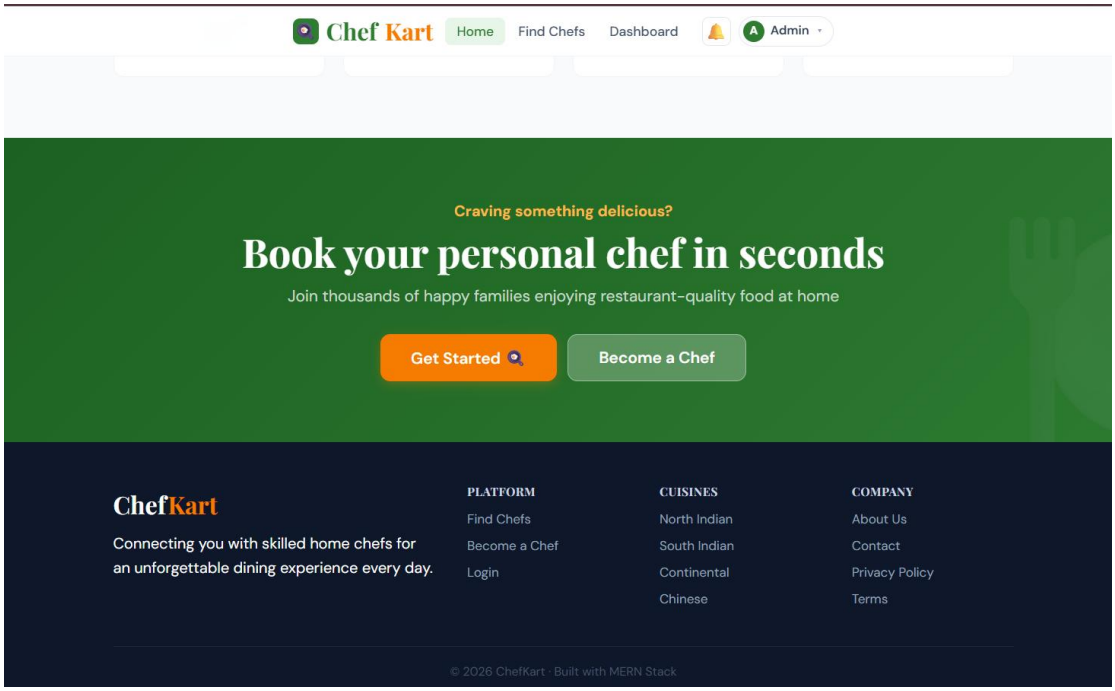
Future enhancements for ChefKart could include real-time notifications via WebSockets, a mobile application (React Native), subscription packages for frequent customers, multi-language support, a loyalty rewards system, AI-based chef recommendation engine, and integration with food delivery logistics for broader service offerings.

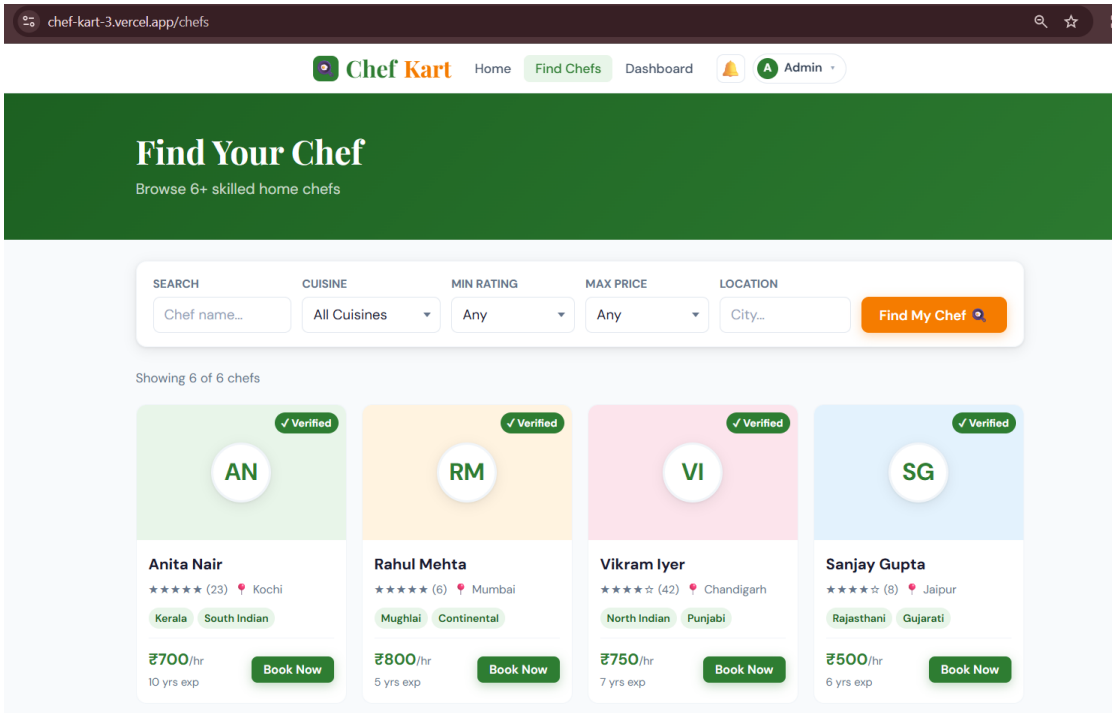
Website Screenshot –




Chef Kart
Home
Find Chefs
Dashboard
Admin
Top Chefs Near You
Handpicked professional chefs for you
View all chefs →
Top Rated
✓ Verified
AN
Anita Nair
★★★★★ (23) Kochi
Kerala
South Indian
₹700/hr
10 yrs exp
Book Now
RM
Rahul Mehta
★★★★★ (6) Mumbai
Mughlai
Continental
₹800/hr
5 yrs exp
Book Now
VI
Vikram Iyer
★★★★☆ (42) Chandigarh
North Indian
Punjabi
₹750/hr
7 yrs exp
Book Now
SG
Sanjay Gupta
★★★★☆ (8) Jaipur
Rajasthani
Gujarati
₹500/hr
6 yrs exp
Book Now
PS
MP


Chef Kart
Home
Find Chefs
Dashboard
Admin
Why Choose ChefKart?
Home-cooked Hygienic Food
Fresh, safe & prepared in clean kitchens with quality ingredients
Verified Professional Chefs
Experienced, skilled & background checked for your safety
Instant Booking
Book in minutes & get a chef at your doorstep, same day possible
₹
Affordable Pricing
Great food at prices that fit your budget, starting from ₹300/hr
Craving something delicious?
Book your personal chef in seconds
Join thousands of happy families enjoying restaurant-quality food at home
Get Started
Become a Chef


Chef Kart
Home
Find Chefs
Dashboard
Admin
Craving something delicious?
Book your personal chef in seconds
Join thousands of happy families enjoying restaurant-quality food at home
Get Started
Become a Chef
ChefKart
Connecting you with skilled home chefs for an unforgettable dining experience every day.
PLATFORM
Find Chefs
Become a Chef
Login
CUISINES
North Indian
South Indian
Continental
Chinese
COMPANY
About Us
Contact
Privacy Policy
Terms
© 2026 ChefKart · Built with MERN Stack


chef-kart-3.vercel.app/chefs
Chef Kart
Home
Find Chefs
Dashboard
Admin
Find Your Chef
Browse 6+ skilled home chefs
SEARCH
Chef name...
CUISINE
All Cuisines
MIN RATING
Any
MAX PRICE
Any
LOCATION
City...
Find My Chef
Showing 6 of 6 chefs
✓ Verified
AN
Anita Nair
★★★★★ (23) Kochi
Kerala
South Indian
₹700/hr
10 yrs exp
Book Now
✓ Verified
RM
Rahul Mehta
★★★★★ (6) Mumbai
Mughlai
Continental
₹800/hr
5 yrs exp
Book Now
✓ Verified
VI
Vikram Iyer
★★★★☆ (42) Chandigarh
North Indian
Punjabi
₹750/hr
7 yrs exp
Book Now
✓ Verified
SG
Sanjay Gupta
★★★★☆ (8) Jaipur
Rajasthani
Gujarati
₹500/hr
6 yrs exp
Book Now

Chef Kart | Home | Find Chefs | Dashboard | Admin

Administrator
Admin

- Overview
- Chef Applications 2
- Customers
- Chefs
- Bookings
- Payments

Platform Overview

Real-time metrics across ChefKart

4	6	2	0	2	₹0.0L
Customers	Active Chefs	Total Bookings	Pending	Applications	Revenue

2 Chef Application(s) Pending Review
Review and approve new chef applications
Review Now →

Recent Bookings

CUSTOMER	CHEF	DATE	AMOUNT	PAYMENT	STATUS
b	Anita Nair	2/5/2026	₹1400	pending	confirmed

Chef Kart | Home | Find Chefs | Dashboard | Admin

Administrator
Admin

- Overview
- Chef Applications
- Customers
- Chefs
- Bookings
- Payments

Manage Customers

4 registered customers

NAME	EMAIL	PHONE	JOINED	STATUS	ACTION
b	bl@gmail.com	12334	1/5/2026	Active	Block
A	b@gmail.com	12345	1/5/2026	Active	Block
a	al2@gmail.com	12344	30/4/2026	Active	Block
John Doe	user@chefkart.com	9876543210	30/4/2026	Active	Block

Chef Kart | Home | Find Chefs | Dashboard | Admin

Administrator
Admin

- Overview
- Chef Applications
- Customers
- Chefs
- Bookings
- Payments

Manage Chefs

6 active chefs — you can remove or suspend chefs

CHEF	EMAIL	LOCATION	RATING	PRICE/HR	STATUS	ACTIONS
Vikram Iyer	vikram@chefkart.com	Chandigarh	★ 4.4	₹750	Available	Suspend Remove
Meena Patel	meena@chefkart.com	Delhi	★ 3.5	₹650	Available	Suspend Remove
Sanjay Gupta	sanjay@chefkart.com	Jaipur	★ 3.6	₹500	Available	Suspend Remove
Anita Nair	anita@chefkart.com	Kochi	★ 4.8	₹700	Available	Suspend Remove
Rahul Mehta	rahul@chefkart.com	Mumbai	★ 4.8	₹800	Available	Suspend Remove
Priya Sharma	priya@chefkart.com	Bangalore	★ 3.6	₹600	Available	Suspend Remove

---

Chef Kart | Home | Find Chefs | Dashboard | Admin

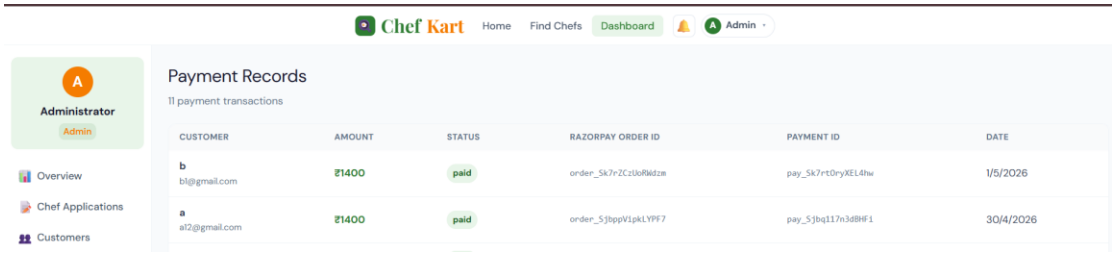
Administrator
Admin

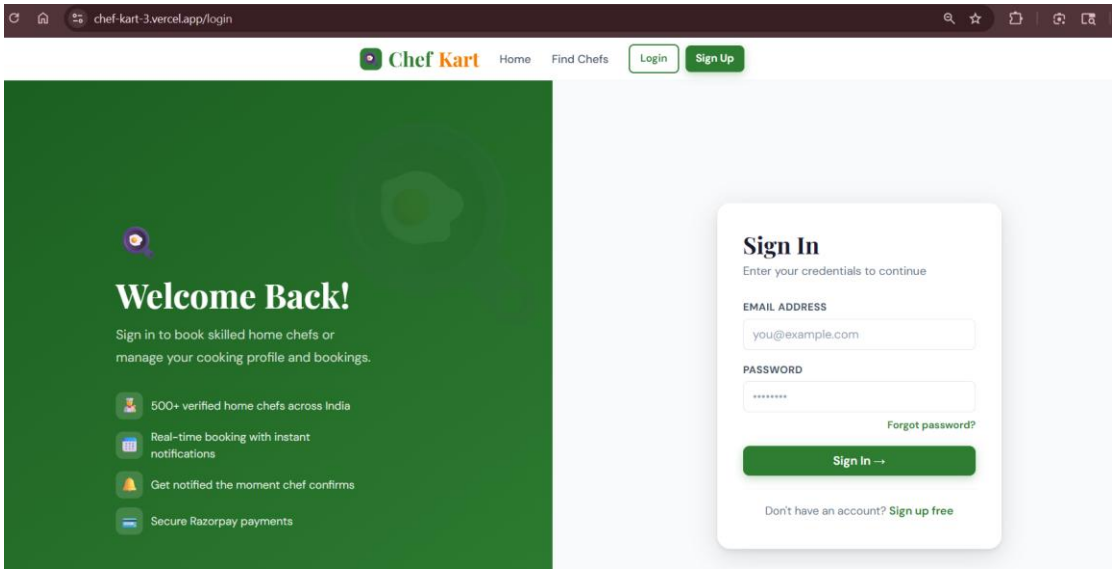
- Overview
- Chef Applications
- Customers
- Chefs
- Bookings
- Payments

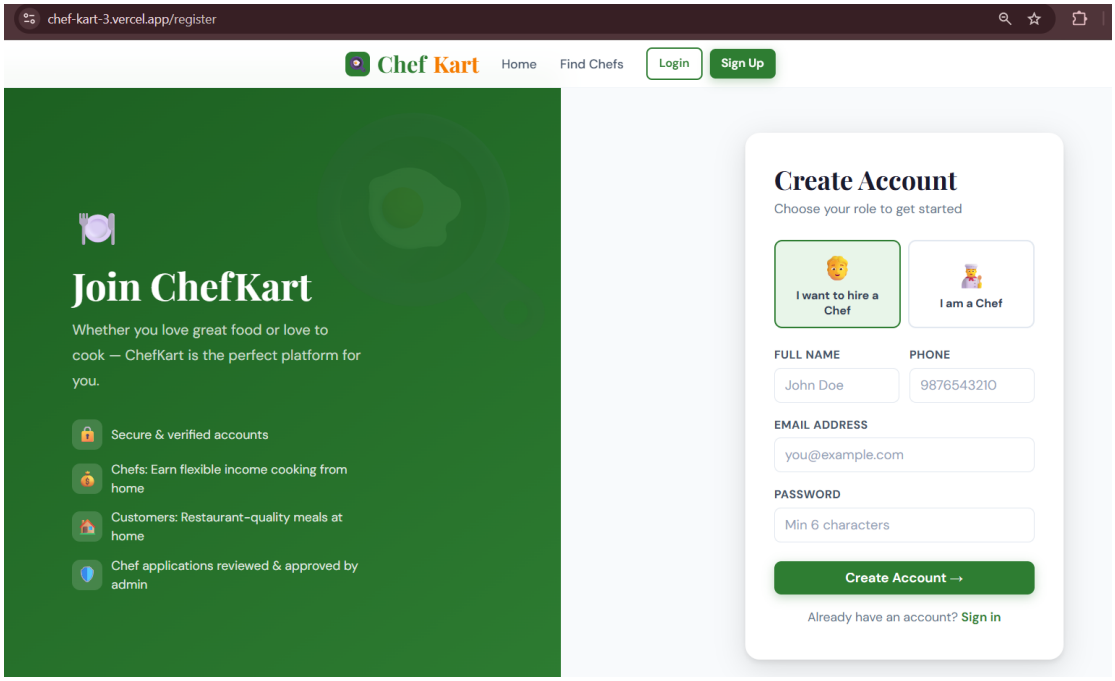
All Bookings

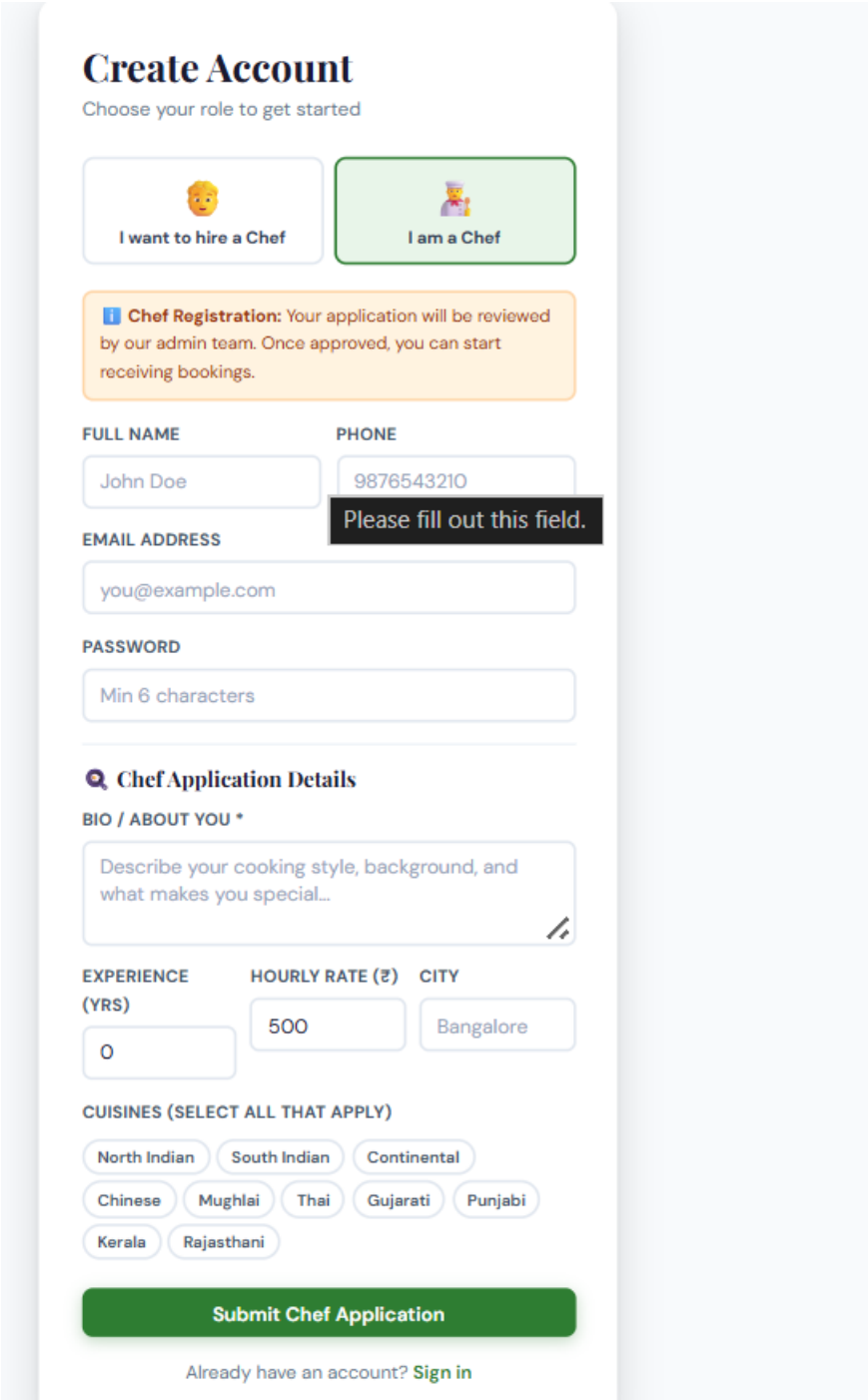
2 total bookings

CUSTOMER	CHEF	DATE	TIME	DUR	AMOUNT	PAYMENT	STATUS
b bl@gmail.com	Anita Nair	2/5/2026	12:00 PM	2h	₹1400	paid	confirmed
a al2@gmail.com	Anita Nair	30/4/2026	04:00 PM	2h	₹1400	paid	confirmed

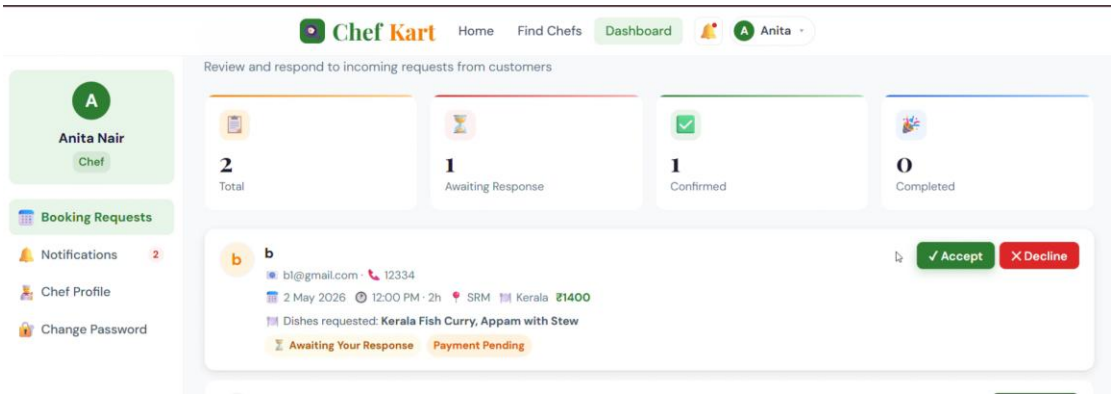

Chef Kart
Home
Find Chefs
Dashboard
Admin
A
Administrator
Admin
Overview
Chef Applications
Customers
Payment Records
11 payment transactions
CUSTOMER
AMOUNT
STATUS
RAZORPAY ORDER ID
PAYMENT ID
DATE
b
bl@gmail.com
₹1400
paid
order_Sk7rZCzUoRWdzm
pay_Sk7rtOryXEL4hw
1/5/2026
a
al2@gmail.com
₹1400
paid
order_SjbppVipkLYPF7
pay_Sjbq117n3d8HF1
30/4/2026


chef-kart-3.vercel.app/login
Chef Kart
Home
Find Chefs
Login
Sign Up
Welcome Back!
Sign in to book skilled home chefs or manage your cooking profile and bookings.
500+ verified home chefs across India
Real-time booking with instant notifications
Get notified the moment chef confirms
Secure Razorpay payments
Sign In
Enter your credentials to continue
EMAIL ADDRESS
you@example.com
PASSWORD
Forgot password?
Sign In →
Don't have an account? Sign up free

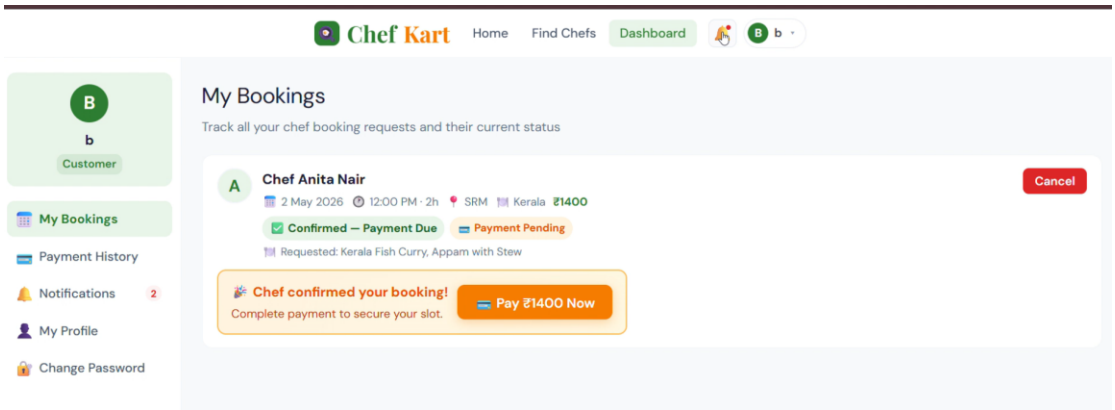

chef-kart-3.vercel.app/register
Chef Kart
Home
Find Chefs
Login
Sign Up
Join ChefKart
Whether you love great food or love to cook — ChefKart is the perfect platform for you.
Secure & verified accounts
Chefs: Earn flexible income cooking from home
Customers: Restaurant-quality meals at home
Chef applications reviewed & approved by admin
Create Account
Choose your role to get started
I want to hire a Chef
I am a Chef
FULL NAME
John Doe
PHONE
9876543210
EMAIL ADDRESS
you@example.com
PASSWORD
Min 6 characters
Create Account →
Already have an account? Sign in

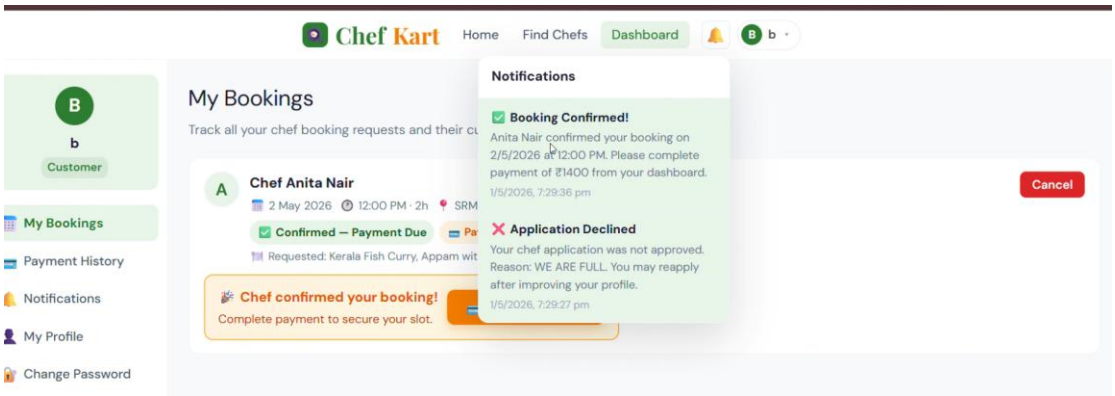


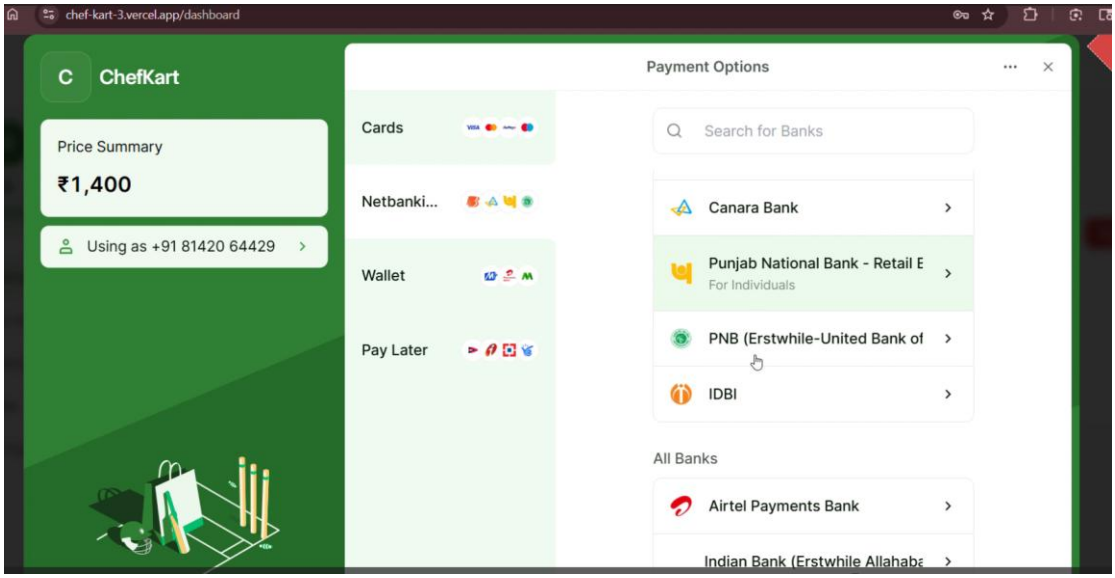
Chef dashboard -

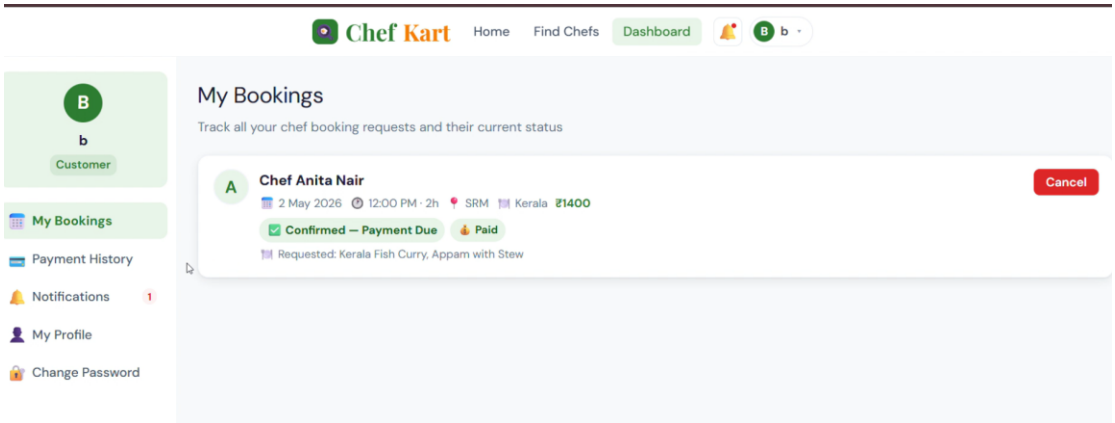


User Dashboard after chef accepts the request -









References & Resources

- GitHub Repository: https://github.com/Amrutha123-bot/ChefKart
- Live deployed application link: https://chef-kart-3.vercel.app/